

Cloud Database-as-a-Service (DaaS) - ROI

V. Mateljan

Faculty of Humanities and Social Sciences
University of Zagreb
I. Lučića 3, Zagreb, Croatia
Tel: 385 1 6002319
Fax: 385 1 6156879
e-mail: vladimir.mateljan@ffzg.hr

D. Čišić

Faculty of Maritime Studies
University of Rijeka
Studentska ulica 2, Rijeka, Croatia
Tel: 385 51 338 411
Fax: 385 51 336 755
e-mail: dragan@pfri.hr

D. Ogrizović

Faculty of Maritime Studies
University of Rijeka
Studentska ulica 2, Rijeka, Croatia
Tel: 385 51 338 411
Fax: 385 51 336 755
e-mail: dario@pfri.hr

Abstract – Cloud computing concept has been presented with a overview of main services, cloud native and cloud capable relational / non-relational databases, and some advances and drawbacks of cloud Database-as-a-Service. Proposed Return On Investment (ROI) analysis outlines how to decide and the things to consider whether an application is suited for a cloud computing environment, on-premise (in-house) infrastructure or to outsourcing to managed services.

I. INTRODUCTION

Cloud computing is an increasingly popular technology for accessing computing resources using network connections. There are lots of cloud computing definitions [1], [2], [3] but the most accepted definition is from USA NIST (National Institute of Standards and Technology), which lays out five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service. Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. [4]

Physical resources, infrastructure, middleware platforms and applications are being provided and consumed as services in the cloud computing. Main service models include:

Infrastructure as a service (IaaS) displaces in-house servers, storage and networks by providing those resources on-demand. Instead of purchasing a server, users can now provision one within minutes and discard it when they are finished, often paying by the hour only for what they actually used.

Platform as a service (PaaS) adds a layer to the infrastructure, providing a platform upon which applications can be written and deployed.

These platforms aim to focus the programmers on the business logic, freeing them from the worries of the physical (or virtual) infrastructure.

Software as a service (SaaS) refers to applications running on cloud infrastructures, typically delivered to the end user via a web browser.

Cloud Database-as-a-Service (DaaS) hosts databases in the cloud environment and provides database features such as data definition, storage and retrieval, on a subscription basis over the Internet. Main cloud computing providers (Amazon, Google, IBM, Microsoft, Oracle) are targeting the DaaS market with their cloud DaaS solutions. DaaS solutions started as very simple, such as Amazon

SimpleDB a key-value type of database, but more sophisticated offerings evolved to support larger and more complex applications, such as Amazon Relational Database Service (RDS) with the full features and capabilities of a relational database.

II. DATABASES IN CLOUD

Cloud computing providers have a number of database alternatives for developers which can be summed into three groups. Cloud "native" relational DBMS enables a fully featured relational database while offloading infrastructure management and database administration to cloud computing providers with flexibility to scale database compute and storage resources with an API call, and only to pay for the consumed infrastructure resources. Cloud "native" non-relational databases provide simple index and query capabilities without administration and with seamless scalability up or down in response to demand, without user intervention. Cloud capable relational / non-relational databases using virtual machines with various operating systems and DBMS to avoid the infrastructure and hardware provisioning typically associated with installing a new database server, but with a complete control over the administrative and tuning tasks associated with running a database server.

A. Amazon Web Services (AWS)

The most prominent cloud computing provider today is Amazon with its Amazon Web Services (AWS). Amazon not only offers the most complete stack of services, but makes it especially easy to integrate different services. [5]

There are three database solutions available at AWS:

Amazon Relational Database Service (RDS) for when the application requires a relational database but users want to reduce the time they spent on database management, Amazon RDS automates common administrative tasks to reduce users complexity and total cost of ownership. Amazon RDS allows users to manage their database compute and storage resources with a simple API call, and only pay for the infrastructure resources they actually consume. [6]

Amazon EC2 - Relational Database Amazon Machine Image (AMI) for when the application requires the use of a particular relational database and/or when the customer wants to exert complete administrative control over their database. An Amazon EC2 instance can be used to run a database, and the data can be stored within an Amazon Elastic Block Store (Amazon EBS) volume. Amazon EBS

is a fast and reliable persistent storage feature of Amazon EC2. Available AMIs include IBM DB2, IBM Informix Dynamic Server, Microsoft SQL Server, MySQL, Oracle, PostgreSQL, Sybase, and Vertica. [7]

Amazon SimpleDB for applications that do not require a relational model, and that principally demand index and query capabilities. Amazon SimpleDB eliminates the administrative overhead of running a highly-available production database, and is unbound by the strict requirements of a RDBMS. With Amazon SimpleDB, you store and query data items via simple web services requests, and Amazon SimpleDB does the rest. In addition to handling infrastructure provisioning, software installation and maintenance, Amazon SimpleDB automatically indexes your data, creates geo-redundant replicas of the data to ensure high availability, and performs database tuning on customers' behalf. Amazon SimpleDB also provides no-touch scaling. There is no need to anticipate and respond to changes in request load or database utilization; the service simply responds to traffic as it comes and goes, charging only for the resources consumed. [8]

B. Microsoft SQL Azure

Microsoft SQL Azure Database is a cloud-based relational database service built on SQL Server technologies. It provides a highly available, scalable, multi-tenant database service hosted in the cloud. SQL Azure Database helps to ease provisioning and deployment of multiple databases. Developers do not have to install, setup, patch or manage any software. High availability and fault tolerance is built-in and no physical administration is required. SQL Azure Database supports Transact-SQL (T-SQL). Customers can use existing knowledge in T-SQL development and a familiar relational data model for symmetry with existing on-premises databases. SQL Azure Database can help reduce costs by integrating with existing toolsets and providing symmetry with on-premises and cloud databases. [9]

C. Google AppEngine Datastore

Google AppEngine Datastore is built on BigTable [3], Google's internal storage system for handling structured data. In and of itself, the AppEngine Datastore is not a direct access mechanism to BigTable, but can be thought of as a simplified interface on top of BigTable.

The AppEngine Datastore supports much richer data types within items than SimpleDB, including list types, which contain collections within a single item. [11]

D. FathomDB

FathomDB allows companies to store their database on Amazon's or Rackspace's servers which offers the benefits of the MySQL database while also taking advantage of the cloud-computing infrastructure to scale up (combine the power of multiple servers) and down (shrink to use only a fraction of a machine) as needed. FathomDB offers a range of time-saving features, including automated backups and routine maintenance tasks, powerful analytic tools, real-

time monitoring and performance graphs for optimizing code, and easy point-and-click configuration. [12]

E. Non-relational databases (NoSQL)

Next Generation Databases mostly addressing some of the points: being non-relational, distributed, open-source and horizontal scalable. The original intention has been modern web-scale databases. The movement began early 2009 and is growing rapidly. Often more characteristics apply as: schema-free, easy replication support, simple API, eventually consistent and the ability to scale in cloud environments. They are collectively called NoSQL or non-relational databases. [13] [14]

Core NoSQL systems include wide column store / column families, document store, key value / tuple store, eventually consistent key value store and graph databases.

List of main NoSQL databases:

Hypertable, modeled after Google's Bigtable project, is designed to manage the storage and processing of information on a large cluster of commodity servers, providing resilience to machine and component failures. Hypertable seeks to set the open source standard for highly available, petabyte scale, database systems. [15]

Apache CouchDB is a document-oriented database that can be queried and indexed in a MapReduce fashion using JavaScript. CouchDB also offers incremental replication with bi-directional conflict detection and resolution. [16]

Mongo is a document-oriented JSON database, except that it is designed to be a true object database, rather than a pure key/value store. MongoDB bridges the gap between key-value stores (which are fast and highly scalable) and traditional RDBMS systems (which provide rich queries and deep functionality). [17]

The Apache Cassandra Project develops a highly scalable second-generation distributed database, bringing together Dynamo's fully distributed design and Bigtable's ColumnFamily-based data model. [18]

Project Voldemort is a distributed key/value database that is intended to scale horizontally across a large numbers of servers. It spawned from work done at LinkedIn and is reportedly used there for a few systems that have very high scalability requirements. Project Voldemort also uses an eventual consistency model, based on Amazon's. [19]

HBase is an open-source, distributed, column-oriented store modeled after Google's Bigtable. Just as Bigtable leverages the distributed data storage provided by the Google File System, HBase provides Bigtable-like capabilities on top of Hadoop. [20]

There are several similar cloud technologies to support DaaS development like Zoho [21], Salesforce [22] and LongJump [23] which implemented "multi-tenant" hosting of many independent schemas in a single managed DBMS.

Table 1 compares cloud "native" and cloud capable relational / non-relational databases.

TABLE 1: COMPARISON OF RELATIONAL AND NON-RELATIONAL CLOUD “NATIVE”/ CLOUD CAPABLE DATABASES

	Cloud “Native”	Cloud capable
Relational	Amazon Relational Database Service (RDS) FathomDB Microsoft SQL Azure	IBM DB2, IBM Informix Dynamic Server, Microsoft SQL Server, MySQL, Oracle, PostgreSQL, Sybase, Vertica, ...
Non-Relational	Amazon SimpleDB Google AppEngine Datastore	Hypertable, MongoDB, HBase, Apache CouchDB, Apache Cassandra, Project Voldemort, ...

III. DAAS ADVANTAGES / DISADVANTAGES

Database-as-a-Service (DaaS) provides the ability to leverage the services of a remotely hosted database, sharing it with other users and having it logically function as if the database were local. Users can self-provision a database, create the tables, load the data, and access the data using the interface provided, all on demand and using cloud computing (figure 1.).

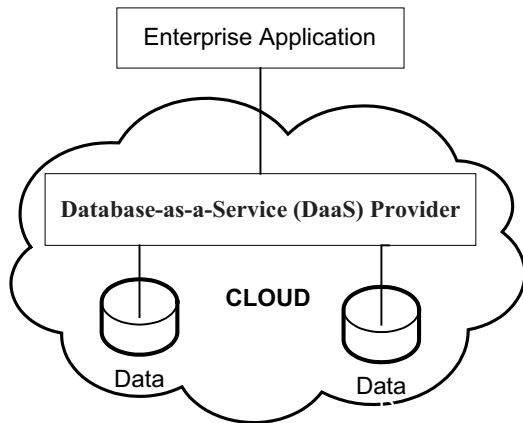


Figure 1: Cloud Database-as-a-service (DaaS)

A. Advantages of DaaS

Advantages include the ability to avoid hardware and software costs (database licensing costs) by leveraging a remote database that users can use when they need it and just what they need to use and therefore users are paying for resources when they use it. All resources, including expensive networking equipment, servers, IT personnel, etc. are shared, resulting in reduced costs. Dynamic scalability or the ability to provision and decommission servers on demand, but can also be a disadvantage because the majority of traditional database servers are incapable of satisfying this requirement. Database maintenance, including backing up and restoring the database, managing users, upgrades and bug fixes to the database are done by

DaaS provider. Instead users can focus on database design and usage.

B. Disadvantages of DaaS

When sharing of physical resources in a cloud infrastructure, data security and privacy cannot be guaranteed by physical boundaries of machines or networks. [24], [25], [26] Interfaces offered by DaaS providers are proprietary in nature and thus can be difficult to leverage from applications that need to access the data so users need to understand and test their interfaces and/or APIs. Some DaaS providers offer only a subset of the capabilities found in traditional on-premise enterprise databases. For example, stored procedures and triggers may not be supported in the same manner as in on-premise databases. Scalability, traditional databases do not scale effectively to thousands of nodes being deployed in a massively shared infrastructure though advance planning must be applied [27]

IV. ROI ANALYSIS

Business decisions are fundamentally focused on increasing revenue while minimizing costs. Return On Investment (ROI) calculations are a tool used by executives to plan investments to achieve this balance, or to ensure that prior investments are performing according to expectation. Although the principle of ROI calculation is simple, the purpose of this analysis is not to provide a definitive ROI methodology for the cloud DaaS versus on-premises (in-house) database servers, but instead to outline how to look at the things users must consider when setting up such an analysis. User analysis should include the purchase decisions they would make in building an infrastructure and the costs of those decisions to their business.

First thing users need to decide is whether an application and therefore database is suited for a cloud computing environment. When application is idling most of the time so it can be shut down when the work is done (hourly provision), when application is growing fast and application with unpredictable or predictable bursts are cases where new infrastructure is needed online fast and to have automatic excess capacity available in the cloud is better solution than having to build it in own data center or leasing another server from hosting provider. When application has a static demand pattern (the same steady amount of usage) users won't see most of the key cost benefits of being in the cloud computing environment.

In ROI analysis cloud DaaS need to be compared against two approaches, to on-premise (in-house) infrastructure and to outsourcing to managed services.

The most appropriate ROI formula (1) for evaluating IT investments is net project benefits divided by cost. The ratio is usually expressed as a percentage when the fractional values are multiplied by 100. Net benefits are project benefits minus project costs. [28]

$$ROI (\%) = \frac{\text{Net Project Benefits}}{\text{Project Costs}} \times 100 \quad (1)$$

When defining project benefits in cloud computing, benefits are cloud DaaS costs compared to on-premise (in-

house) infrastructure cost. Cloud DaaS costs include estimated costs of virtual server or pricing is per DB Instance/hour, backup storage and data transfer, estimated licensing fees, costs for cloud infrastructure management tools, labour costs for creating machine images, managing the infrastructure, and responding to issues over three years. On-premise (in-house) infrastructure cost include servers, networking equipment and software licenses, real estate, energy, cooling, IT admin staffs, management and monitoring software and servers and on and on. Managed services outsourcing has similar benefits to the cloud in that users pay a fixed fee for someone else to manage their leased servers. Main difference between cloud DaaS and managed services are increased agility (the ability to get up and running with new products and services faster), with the ability to scale on-demand to handle unexpected loads (and thus prevent loss of business due to decreased performance or outright downtime).

Microsoft has an online calculator for determining ROI as well as total cost of ownership (TCO) of deploying its new Windows Azure Platform (including Microsoft SQL Azure) for doing enterprise computing in the cloud. [29]

Amazon has an online calculator to give their customers an opportunity to estimate their costs. This calculator incorporates a wide array of pricing calculations across all Amazon Web Services in all their regions. It also shows breakdown of features for each service in each region. [30]

The ROI on cloud DaaS, in general, is determined by how much more agile the company can be, how much quicker it can bring products to market and how much sooner it can launch other initiatives to bring in more revenue and generate more profit.

V. CONCLUSION

DaaS enabled instant on-demand network access to configurable computing resources where users can avoid over-provisioning and under-provisioning, to improve cost, revenue and to provide new business services based on new ways of operating.

Proposed ROI analysis outline how to look at the things users must consider when setting up such an analysis. It will help to decide whether an application is suited for a cloud computing environment, on-premise (in-house) infrastructure or to outsourcing to managed services.

As cloud DaaS become more popular, new scenarios will emerge with their own challenges.

REFERENCES

- [1] B. Hayes, "Cloud computing", *Communications of the ACM*, (7):9–11, 2008.
- [2] J. Geelan, "Twenty one experts define cloud computing", *Electronic Magazine*, 2008.
- [3] G. Gruman and E. Knorr, "What cloud computing really means", *InfoWorld*, 2008.
- [4] P. Mell and T. Grance, "The NIST Definition of Cloud Computing", <http://csrc.nist.gov/groups/SNS/cloud-computing/> (accessed 12.01.2010)
- [5] Amazon Web Services (AWS), <http://aws.amazon.com/>, (accessed 10.01.2010)
- [6] Amazon EC2 - Relational Database Amazon Machine Image (AMI), http://aws.amazon.com/running_databases/#relational, (accessed 10.01.2010)
- [7] Amazon Relational Database Service (RDS), <http://aws.amazon.com/rds/>, (accessed 10.01.2010)
- [8] Amazon SimpleDB, <http://aws.amazon.com/simplydb/>, (accessed 10.01.2010)
- [9] Microsoft SQL Azure, <http://www.microsoft.com/windowsazure/sqlazure/>, (accessed 14.01.2010)
- [10] BigTable, <http://labs.google.com/papers/bigtable.html>, (accessed 20.01.2010)
- [11] Google AppEngine Datastore, <http://code.google.com/appengine/>, (accessed 15.01.2010)
- [12] FathomDB, <http://fathomdb.com>, (accessed 29.01.2010)
- [13] NoSQL, <http://en.wikipedia.org/wiki/NoSQL>, (accessed 22.01.2010)
- [14] NoSQL, <http://nosql-database.org/>, (accessed 22.01.2010)
- [15] HyperTable, <http://hypertable.org/>, (accessed 20.01.2010)
- [16] Apache CouchDB, <http://couchdb.apache.org/>, (accessed 20.01.2010)
- [17] MongoDB, <http://www.mongodb.org/>, (accessed 26.01.2010)
- [18] Apache Cassandra, <http://cassandra.apache.org/>, (accessed 20.01.2010)
- [19] Project Voldemort, <http://project-voldemort.com/>, (accessed 26.01.2010)
- [20] HBase, <http://hadoop.apache.org/hbase/>, (accessed 26.01.2010)
- [21] Zoho, <http://www.zoho.com/>, (accessed 27.01.2010)
- [22] Salesforce, <http://www.force.com/>, (accessed 27.01.2010)
- [23] LongJump, <http://www.longjump.com/>, (accessed 27.01.2010)
- [24] E. Mykletun and G. Tsudik, "Incorporating a secure coprocessor in the database-as-a-service model", *IWIA*, 2005.
- [25] S. Pearson, "Taking Account of Privacy when Designing Cloud Computing Services", *CLOUD'09*, 2009.
- [26] W. K. Wong, W. Cheung, B. Kao, and N. Mamoulis, "Secure kNN Computation on Encrypted Databases", *SIGMOD*, 2009
- [27] Z. Wei, G. Pierre and C.H. Chi, "Scalable Transactions for Web Applications in the Cloud", *Proceedings of the EuroPar conference*, 2009.
- [28] B. Roulstone and J.J. Phillips, "ROI for Technology Projects: Measuring and Delivering Value", Elsevier, 2008.
- [29] Microsoft Windows Azure Platform TCO and ROI Calculator, <http://www.microsoft.com/windowsazure/tco/>, (accessed 29.01.2010)
- [30] AWS Simple Monthly Calculator, <http://aws.amazon.com/calculator/>, (accessed 29.01.2010)